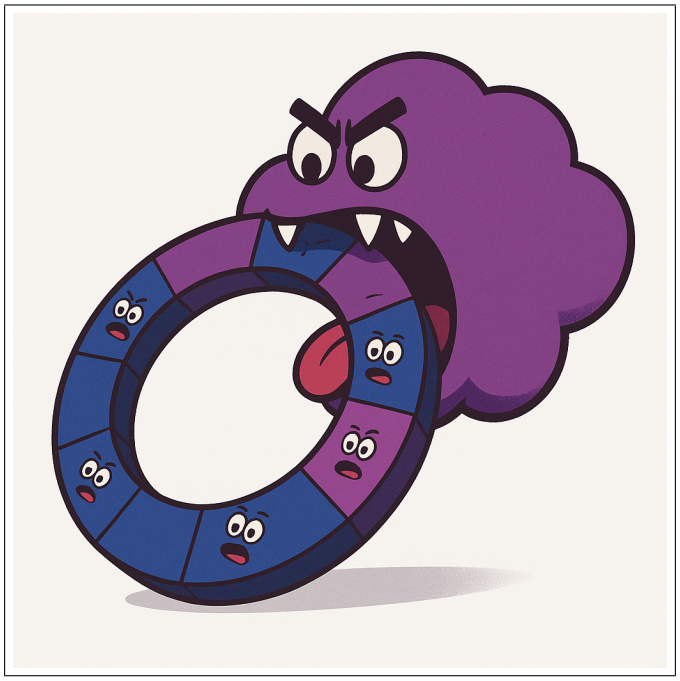
FernUniversität in Hagen

Dokumentation

Programmierpraktikum

B. Sc. Informatik



Name: Vanessa Köbke
Matrikelnummer: 8878390
Lehrgebiet: Kommunikationsnetze
Professor: Prof. Dr. Herwig Unger
Abgabedatum: 04.01.2025

Fakultät für Mathematik und Informatik
FernUniversität in Hagen

Contents

1 Introduction **3**

2 Strategy **3**

2.1 Initial thoughts . . . 3

2.1.1 Adaptability . . . 4

2.1.2 Basic strategies . . . 4

2.1.3 Checks . . . 5

2.1.4 Back up strategies . . . 5

2.2 Literature review . . . 6

2.2.1 Learning from nature . . . 6

2.2.2 Contextualizing within basic AI theory . . . 7

2.3 My agent's strategy . . . 9

3 My agent **10**

3.1 Model . . . 11

3.1.1 Ring . . . 11

3.1.2 Node . . . 12

3.1.3 Output . . . 12

3.1.4 Notes . . . 13

3.1.5 IllegalStatusException . . . 13

3.1.6 MoveException . . . 13

3.2 Service . . . 14

3.2.1 Strategy . . . 14

3.2.2 Analyzer . . . 15

3.2.3 Util . . . 15

4 Performance analysis and the influence of game settings **16**

4.1 Ring size . . . 17

4.2 Fernie growth per turn . . . 17

4.3 Node ownership bonus . . . 17

4.4 Fernies at the beginning of the game . . . 17

4.5 Visibility . . . 18

5 Conclusion **18**

Literature **19**

A Anhang **20**

A.1 Results against Agent2 . . . 20

A.2 Results against itself - MixedStrategy 20
A.3 Results against Agent C 21
A.4 Results against Agent K 22

1 Introduction

The objective of this paper is to describe the RINGwars agent which was developed within the scope of the Programmierpraktikum. RINGwars is a competitive, turn-based game for two players. The goal of the game is to conquer as many nodes on the ring as possible in order to eliminate the opponent.

Section 2 describes the genesis of my agent's strategy. The section starts with my initial thoughts on this project and the requirements I considered relevant for a successful strategy. A review of two types of literature follows: (i) the strategies used by animals (mainly moose and ants) for conquering and securing territory and (ii) a contextualization of these strategies in the field of basic AI. The section closes with the description of my agent's strategy.

Section 3 describes the structure of the Java program which implements my strategy. This section is structured by the program packages and briefly explains the core function of each class, their structure and their most relevant methods.

Section 4 analyzes the influence of different game setting parameters on the performance of my agent, as compared to two exemplary agents by fellow students. Section 5 concludes.

2 Strategy

2.1 Initial thoughts

My initial approach to this task was to reflect on how I, as a human, would try to win the game and try to implement my reasoning in code. The result of this initial reflection were the following basic ideas. I will elaborate on these ideas in the subsections below.

1. **Adaptability:** The strategy for any given turn should depend on the current situation: the status of the ring in that round, and the information available from previous rounds.
2. **Basic strategies:** The agent should possess several basic strategies that can be used individually or in combination. These basic strategies should vary on at least two criteria: aggressiveness and expansiveness.
3. **Checks:** In order to avoid illegal moves, I would need checks and safeguards to detect problems as early as possible.
4. **Backup strategies:** I would need backup strategies, in case the chosen strategy produced illegal moves, the file in which information on previous rounds was stored was corrupted or other unforeseeable situations.

2.1.1 Adaptability

The strategy of my agent should adapt to the circumstances. In order to do that, I first needed to identify the relevant criteria. These criteria would become the forks in my decision tree.

The first aspect that comes to mind is whether the opponent is visible or not. The basic idea is that if my agent cannot see the opponent, the opponent cannot see my agent either. Thus they cannot attack nodes owned by my agent. It is therefore a state in which my agent can safely expand. Given the existence of a node bonus, it is recommendable to own as many nodes as possible[1].

The second aspect is the relative strength of my agent as compared to the opponent. As soon as the full ring is visible, the relative strength can easily be determined by comparing the total number of fernies of my agent and the opponent. When the ring is only partly visible, this could be estimated by the average number of fernies per node of my agent and the opponent. However, this approach does not work if the opponent leaves most of his fernies on invisible nodes or puts most of them on border nodes. It is therefore preferable to avoid using average fernies per node in choosing the strategy.

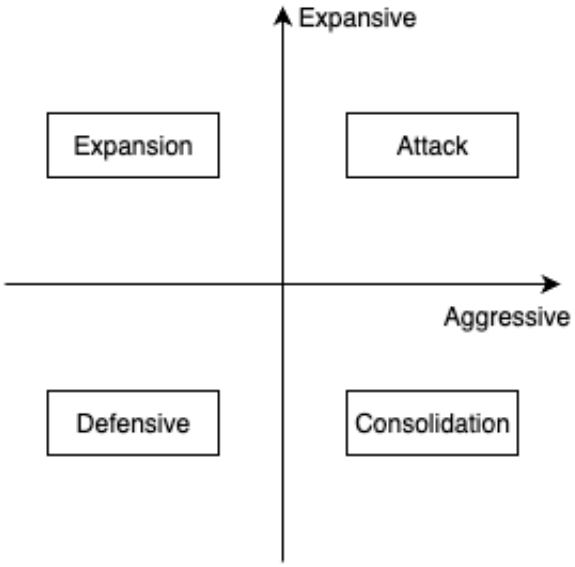
The third aspect is the opponent's strategy. I.e. if the opponent plays very aggressively, it might be smart to play defensively until my agent's relative strength is greater than the opponent's. Alternatively it might be smart to fight back as early as possible, before the opponent becomes to strong. In order to choose a fitting strategy it is necessary to store information on the opponent's strategy in between rounds. In order to do this, I will create a notes file (for more information on its implementation see section 3.1.4) which stores the parameters of the opponent's strategy.

The fourth aspect is the success rate of my agent, i.e. how many of my agent's attacks were successful, how many failed. Tracking the success of my strategies allows to make a more fine-grained adjustment to the strategy. More precisely this allows to create a mixed strategy which combines different approaches.

2.1.2 Basic strategies

I identified two criteria based on which the strategy could be parameterized: aggressiveness and expansiveness. I therefore drafted four basic basic strategies.

[1] However, this fact needs to be put into perspective. If the number of nodes on the ring is small (such as in the default setting) or the number of owned fernies is small, the node bonus hardly has any effect. For example, in the first rounds of the game there is no difference in new fernies whether the agent occupies 10 nodes or only 1 node.



Expansion
A strategy which occupies every unoccupied node.
Attack
A strategy that attacks the opponent's nodes.
Consolidation
A strategy that fortifies nodes that are visible to the opponent.
Defensive
A strategy that abandons nodes that are too weak to defend.

2.1.3 Checks

Since illegal moves (e.g. placing more fernies than the agent possesses) result in a lost turn, they should be avoided at all cost. I therefore need to implement several (perhaps redundant) checks in order to make my program as resilient as possible against unintended moves.

Checks should be implemented in 3 places:

- Output: before the output is printed into the move.txt it should be verified that the agent didn't use more fernies than they had available.
- Ring: Before allowing to carry out a move, it should be verified that no ring properties are violated (e.g. maximum fernies per node).
- Node: Before allowing to carry out a move, it should be verified that no node properties are violated (e.g. removing fernies from a node that does not belong to the agent).

2.1.4 Back up strategies

If the checks show that an illegal move is attempted, the first solution should be to fix it where it happened (see section 3.1.6). However, if this fails it is preferable to carry out a non-optimal strategy that is proven to produce legal moves over losing the turn entirely.

2.2 Literature review

2.2.1 Learning from nature

In nature, species without enemies in the area expand quickly. A famous example of this are the moose of Isle Royale. Isle Royale is an island on Lake Superior in Michigan. In 1905 no moose lived on Isle Royale (Murie 1934, p. 10). The winter of 1912/1913 was so cold that an ice bridge became available for moose to cross over to Isle Royale. Since they had no enemies on Isle Royale, their numbers expanded from just a few to over a thousand in 1930 (ibid.). Assuming that Isle Royale corresponds to the ring, and the moose are fernies, it seems beneficial to expand as much as possible upon arrival/beginning of the game.
Ants present similar behavior. Eldridge and Tschinkel (2001, p. 360) show that in the absence of competitors, fire ants more than double their territory (increase of $122 \pm 52\%$) in 8 weeks.

Ellis and Robinson (2014, p. 116) provide us with an explanation for the rapid expansion strategy of red wood ants: "If nest-site availability is the limiting factor in the environment then it may be advantageous for a colony to become polydomous[2] to take advantage of available nest sites in the vicinity, as an investment in future colony growth." This behavior seems to follow the same intuitive logic one might have about RINGwars: occupying an empty nest (or node) is easier than fighting for a nest/node that is occupied by someone else. The reference to future colony growth can be directly mapped to the node bonus for the fernie growth rate. The same applies to the nest-site availability as being the limiting factor; this is equivalent to the limited number of nodes on a ring. Both moose and ants behavior seem to indicate that expanding while no competitor is in sight is beneficial.

The tragic sequel of the moose on Isle Royal, however, was that due to depletion of food resources and the immigration of wolves their numbers plummeted (Isle Royale Wolf-Moose Project 2024). The resource depletion is irrelevant for RINGwars, since fernies do not need resources to survive[3]. The appearance of a predator, on the other hand, is relevant. Fortunately, our fernies are not as defenseless as moose against wolves, and viable counter-strategies can be found. This can either be a aggressive or defensive strategy based on the current situation.

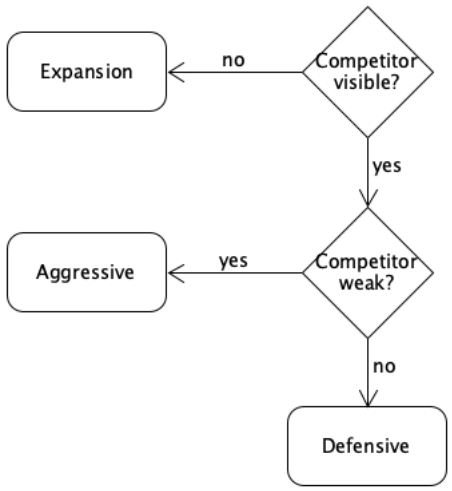
Ants again provide an example of an aggressive strategy for situations when the opponent is still weak. Adams (2016, p. 107) states ants behavior as follows: "Territorial ants kill queens and incipient colonies of their own species [...]. Although a nest-founding queen is not herself much of a threat [...], she is the seed from which a formidable competitor may grow." If we see the queen as being equivalent to the opponent's starting node (or later in game, their strongest node), the strategy that can be derived from ants' strategy is to attack the

[2]Polydomous means to have several nests. (My addition)
[3]The worst that can happen is that fernie numbers stagnate at the maximum fernie number per node.

"queen node" as early as possible. This cripples the enemy in two ways. First, if they lose the majority of fernies, in the next round they will likely be too weak to attack. Secondly, the current number of fernies in an agent's possession directly influences the growth rate in the next round through the fernie bonus. Therefore attacking the strongest node will also lower the opponent's growth rate in future rounds.

Ants equally provide examples of defensive strategies when they are weaker than a competitor. First of all, having multiple nests (nodes) "could also improve colony survival by spreading risks" (Ellis and Robinson 2014, p. 117), i.e. by distributing the fernies more or less evenly over multiple nodes, the risk of losing many fernies at once is reduced. Another strategy ants use when they are attacked is to "abandon the nests, retreating to a different nest in the same colony" (ibid.). In RINGwars terms, it can be beneficial to abandon nodes (despite losing node bonus by doing so) in order to save fernies from opponent attacks by moving them to other nodes that are invisible to the opponent or that are more fortified.

From the observation of ants' patterns of behavior, the following simple decision tree can be derived.



2.2.2 Contextualizing within basic AI theory

In this section I will elaborate on the AI-related concepts that can be used to refine my basic strategies. RINGwars appears to be a partially observable, multiagent, competitive, deterministic (but unpredictable due to other agent's actions) game (Russell and Norvig 2010, pp. 42–44). Especially the partial observability and the unpredictability of the opponent agent seem to be relevant.

We can see that an ideal agent for RINGwars needs to fulfill some of the requirements usu-

ally considered the basic AI disciplines: knowledge representation, automated reasoning, and machine learning (Russell and Norvig 2010, p. 2).

- The agent should be **rational**, i.e. it should act "so as to achieve the best outcome or, when there is uncertainty, the best expected outcome" (Russell and Norvig 2010, p. 2).
- The agent should be capable of **learning** from its moves in previous rounds.
- The agent should be able to **store** its knowledge.

However, it should be noted that "rationality maximizes expected performance" (Russell and Norvig 2010, p. 38), perfection in the actual outcome cannot be guaranteed in cases of partial observability or an unpredictable opponent player such as in RINGwars. Furthermore my agent cannot be fully rational, since rationality requires autonomy, in the sense that it only relies on its own percepts and not on the prior knowledge of its designer (ibid. 39). Since the instruction for this project was to stick to basic AI, a fully rational agent does not seem to be feasible within the scope of this module. I will, however, follow the idea of Russell and Norvig (2010, p. 39) as they state:

> *So, just as evolution provides animals with enough built-in reflexes to survive long enough to learn for themselves, it would be reasonable to provide an artificial intelligent agent with some initial knowledge as well as an ability to learn. After sufficient experience of its environment, the behavior of a rational agent can become effectively independent of its prior knowledge.*

This leads to the different categories of AI agents. For the purpose of my agent I consider the simple reflex agent and the learning agent the most relevant. Simple reflex agents are governed by condition-action rules (or if-then rules) (Russell and Norvig 2010, p. 48); if a certain condition is met, a predefined series of instructions is carried out. The benefit of this kind of agent is its simplicity and that desired behavior for certain situations can be hard coded. The most relevant difference learning agents present as compared to other models is their ability to observe their behavior and its outcome and adapt future behavior in consequence. This relates directly to the necessity of being able to store knowledge as mentioned above.

Another relevant concept is the idea of a belief state (Russell and Norvig 2010, p. 138). In case of partial observability and an unpredictable opponent, the agent cannot predict the outcome of a turn with certainty. It can however formulate a belief state, i.e. the state of the ring after a turn as the agent believes it to be. Since the number of states that are possible if factoring in all possible moves by the opponent is too vast to consider, for the sake of simplicity my agent will adopt the naive vision that all its moves are successful. At the start of each turn my agent will compare its belief state from the end of the previous turn (stored in a prediction

file) with the actual state of the ring. The agent's pattern of behavior is therefore (i) predict (ii) update (iii) move as in Russell and Norvig (2010, p. 146).

2.3 My agent's strategy

First, I will describe the task environment for my agent (Russell and Norvig 2010, p. 40). The environment in which the agent acts is defined by the status of the ring (and its nodes) and the opponent's behavior. The percepts of the agent (Russell and Norvig 2010, p. 34) are the status files it has access to, as well as its stored knowledge. The agent then maps the percepts onto a move (ibid. , p. 35). The agent's sensor is its readStatusFile and readNotes methods and its actuator is its writeMove and writeNotes methods. The agent's goal is to eliminate the opponent from the ring. Its performance can therefore be measured primarily by the elimination of opponent nodes. Supporting the agent's success are the number of fernies (fernie bonus) and the number of nodes (node bonus) it possesses. Incrementing these numbers is therefore a secondary performance measure.

Performance measure	Environment	Actuators	Sensors
• Eliminating opponent nodes • Occupying as many nodes as possible	• Ring • Opponent	• writeMove • writeNotes	• readStatus • readNotes • readPrediction

As discussed in the previous section, a learning agent (Russell and Norvig 2010, pp. 54–58) appears to be the optimal solution. I will, however, combine it with a simple reflex agent (ibid, p. 48) for situations in which the optimal strategy is clear from the ring status:

- When the opponent is not visible, the agent shall always expand.
- When the opponent is weak, the agent shall always attack.
- When the agent is vastly overpowered, it shall act defensively.

For this purpose my agent possesses four basic strategies that correspond to the ones discussed in sections 2.1 and 2.2.1. In the **Expansion** strategy my agent evenly distributes the available fernies on all visible nodes. This allows to maximize new fernies through the node bonus and optimize visibility for the following turn. In the **Consolidation** strategy the agent removes fernies from the nodes farthest from the opponent (leaving only one fernie on each node) and fortifies the nodes closest to the opponent. The goal of this strategy is to protect the most vulnerable nodes while simultaneously maintaining maximum node bonus.

The **Attack** strategy attempts to conquer nodes from the opponent. Depending on the

strength of the opponent and their number of nodes, there are the AttackMax strategy which attacks the strongest node(s) (see section 2.2.1 about killing the ant queen) and the AttackMin strategy which attacks the weakest nodes. In both cases the agent attempts attacks in the following order. First it attempts triple battle, as this presents the optimal attack if successful: no fernies are lost and the agent possesses three new nodes. Secondly it attempts an edge battle, since this is the second best option: no fernies lost and two new nodes. Only if the previous two strategies fail, the agent carries out a direct attack. A failed attempt in this context means that the agent finds no opponent nodes without neighbors on one or both side and whose fernie count allows for an attack to be successful.

In the **Defensive** strategy the agent vacates all currently held nodes and tries to place all fernies on a node that is invisible to the opponent. If all nodes are visible to the opponent the agent places all fernies on a random uncontrolled node. This element of randomness makes it harder for the opponent to successfully attack my agent.

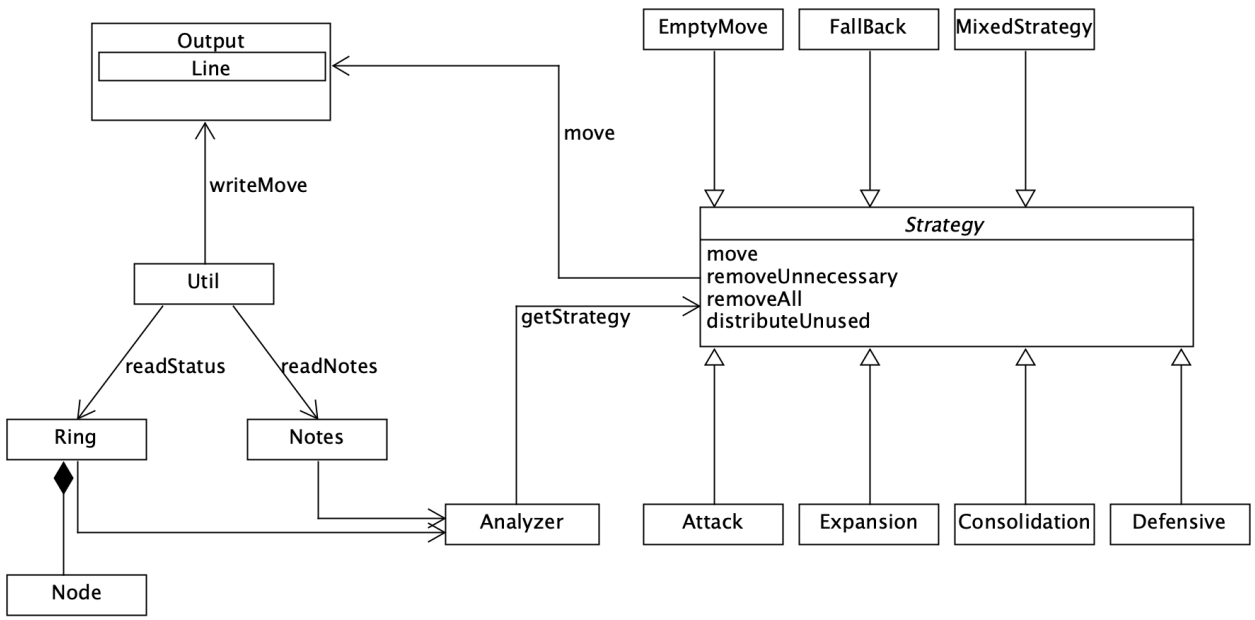
If none of the conditions above are met the learning part of my agent comes into action. At the beginning of each turn the agent analyzes the outcome of the previous turn, more specifically the aggressiveness of the opponent and the success of its own attacks. Depending on these numbers the agent adjusts the ratio of the four basic strategies stored in the notes file. Then the **Mixed Strategy** is executed which distributes the available fernies to the different basic strategies as stated in the ratio.

In order to optimize my strategies I used a simple tracker to count how many games I won and how many I lost (see appendix A for the statistics) and played for each case of settings approx. 100 rounds. As predicted it appears that for the AttackMax strategy it is more efficient to primarily use triple and edge battles (success rate of 78% against Agent2) over local battles (65%). Adding an additional 0.1% attacking buffer for triple and edge battles increases the success rate further to 84%. When trying to determine the ideal starting ratio for the Mixed Strategy I compared more balanced approaches (83% success rate when playing against myself[4]). However, it appears that using only the AttackMin strategy initially provides the best results (94% success rate).

3 My agent

This section describes the key components of my agent. The following diagram gives an simplified overview of the different classes and their interaction.

[4] I played against myself because I defeated Agent2 within few rounds and never reached a point where the Mixed Strategy was used.



3.1 Model

3.1.1 Ring

The Ring class is one of the two most important elements of my program. A Ring object represents the status of the playing field at a given point in time. As its main attributes it contains an array of nodes (see following section) and the numbers of fernies that are currently free and ready to be placed. Additionally it contains two more attributes which represent the constraints placed on the game and the current round, i.e. the maximum number of fernies per node, and the maximum number of fernies that the agent has available this round (sum of new fernies and already placed fernies).

A Ring object is always created by reading the status file of a given round (see section 3.2.3). However, it can be modified by the strategy's methods (particularly the nodes' status and the number of available fernies).

The Ring object based on the current round is used to execute the agent's strategy and the corresponding calculations. A prediction Ring object based on the previous round is used to gain insight on the success of moves and to update the agent's knowledge base (see section 3.1.4).

The Ring class offers a wide variety of methods which can be grouped as follows:

- methods that return information about the ring (e.g. how much of the ring is currently visible)
- methods that return a list of nodes (subset of the ring as a whole) that satisfy a certain

condition (e.g. unoccupied nodes, nodes without neighbors, opponent nodes with the lowest number of fernies, to name just a few)

- methods that execute moves on the ring (e.g. attack opponent node or remove fernies from own node)

The last type of methods include checks on the validity of a given move (requirement 3 from section 2). In case of invalid moves the methods throw a MoveException (see section 3.1.6).

3.1.2 Node

A Node object represents the current status of a node in the game. It contains three attributes that determine the state of a node: the node number, the node's owner and the number of fernies on the node.

The node class has two types of methods:

- methods that return information about the node: which number has the node, who is its owner, how many fernies are placed on the node and is it visible
- methods that execute moves on the node: place fernies and remove fernies

As with the equivalent methods on the ring, when a Node object carries out a place or remove action, it checks the validity of the move. In case of invalid moves the method throws a MoveException (see section 3.1.6).

3.1.3 Output

An Output object represents the moves carried out by the agent in a given round. For this purpose it contains a list of Line objects (Line being an internal class which simply represents one line in the move file) and the maximum number of available fernies for the agent. This last attribute is used to execute the final check (see requirement 3 of section 2) on the validity of the output before returning it as String to be written in the move file.

The three methods of this class are (i) returning the output as a String, (ii) placing fernies on a node, and (iii) removing fernies from a node. The place and remove methods avoid duplicate lines per node number by checking whether a place or remove line already exists for the given node number. If so it adds (or subtracts) the new fernies from the already existing Line object.

3.1.4 Notes

A Notes object represents the knowledge the agent has gathered over the course of the game and the strategy parameters that are to be preserved over the course of several turns. It is a container object and stores the information as attributes. The most relevant attributes are the following:

- current round
- visibility radius
- the aggressiveness and defensiveness of the opponent as length-2 array
- number of my attacks that were blocked by the opponent
- my nodes that were lost to the opponent (last round and over the course of the game)
- attack buffer (i.e. with how many additional fernies my agent attacks)
- ratios of the different substrategies (only relevant to the MixedStrategy)

The methods of the Notes class consist mainly of getters and setters for its attributes.

3.1.5 IllegalStatusException

An IllegalStatusException is thrown when the readStatus method of the Util class reads an inconsistent or implausible step file, e.g. if the node number or status in line 1 and 2 do not match, if the ring is empty or if a NumberFormatException is thrown when trying to parse a fernie number. When an IllegalStatusException is thrown the main method automatically generates an empty move file and prints the error message to the console.

3.1.6 MoveException

A MoveException is used in the following situations: (i) if the agent attempts to carry out an illegal move (e.g. removing fernies from a node that isn't theirs), (ii) if the agent attempts a discouraged move (e.g. attacking an opponent with too few fernies), and (iii) handling fernie overflow. Fernie overflow means that a placement of fernies would lead to the node having more fernies than allowed by the maximum fernie number per node. In this case a FernieException (subclass of the MoveException) is thrown. If a normal MoveException is thrown, the attempted move is aborted and the agent continues with the next move. In contrast, if a FernieException is thrown the attempted move is carried out but only with the number of fernies required to reach maximum fernie count. The exception object contains an attribute with the number of fernies that were actually placed. The calling method then

writes only the used fernies into the output and uses the remaining fernies for other moves.

3.2 Service

3.2.1 Strategy

Strategy is the abstract superclass of all specific strategies. It possesses two attributes (which both have package local visibility instead of private for easier access from the subclasses): a Notes object containing all the gathered information from the current game and the number of fernies that is available for this strategy (only relevant to the MixedStrategy). Furthermore the abstract Strategy superclass contains a series of methods that are used by its subclasses.

1. **move**: This is the main method of all strategies as it receives the current ring as a parameter and returns the output to be written in the move file. In the Strategy superclass this method is abstract because the implementation is different for each subclass and it only represents the interface for the main method.
2. **removeUnnecessary**: This method is usually called upon at the beginning of the move method in the subclasses. It removes all fernies but one from the agent's nodes. The purpose is to guarantee maximum node bonus by retaining as many nodes as possible (with 1 fernie) while also having as many fernies available as possible for the executed strategy.
3. **removeAll**: This method is similar to the previous method, however, it removes all fernies from the agent's nodes. This method is only called upon at the beginning of the Defensive strategy in which the main goal is to preserve as many fernies as possible and to flee from the opponent.
4. **distributeUnused**: This method is called upon at the end of the move method in the subclasses in order to ensure that no available fernies are left unused. This method places all available fernies on the agent's node(s) with the lowest number of fernies until all fernies are used up.

The subclasses of Strategy are Expansion, Consolidation, Attack, and Defensive, and function as described in section 2.3. Attack itself has two subclasses AttackMin and AttackMax whose only difference is whether they attack the opponent node(s) with the highest or lowest number of fernies.

3.2.2 Analyzer

The Analyzer class represents the main logic of my agent. Its only public method is the getStrategy method which returns the strategy to be executed in the current round. Since the Analyzer class has no attributes and there do not need to be any instances of this class, its methods are all static and serve as a kind of utility function.

Internally the getStrategy method is divided into two parts: (i) analysis of the results of the previous round and (ii) strategy selection. During the analysis the ring as it was left by the agent in the previous round is read from the prediction.txt file. Then the predicted ring state and the actual state of the ring are compared. The analyzer subsequently updates the notes with the results. Additionally in certain cases it updates the strategy parameters in the notes; e.g. if the opponent is perceived as more defensive the attack buffer is raised or if the opponent is perceived as more aggressive the defensiveness ratio is raised.

The strategy selection contains two approaches (see section 2.3): (i) the simple reflex agent and (ii) the learning agent. First all conditions of the simple reflex agent are checked. If none apply a MixedStrategy object is returned which uses the learned information from the analysis part.

3.2.3 Util

The Util class provides several static utility methods for reading files (status or notes) and processing their information into objects with which the agent can work, and writing the move, notes and prediction files. In case of reading the status file the Util class checks the validity of the file's content. If the status file is invalid the method throws an IllegalStatusException (see section 3.1.5), which in turn triggers the EmptyMove strategy.

In case of reading the notes file, the Util class parses the information in the notes.txt file and restores the Notes object in its state at the end of the previous turn. In earlier versions of my agent, this method also carried out the analysis of the results of the previous round (recognizing changes in enemy strategy, raising attack buffer etc.). In the final version of my agent the analysis was removed from the Util class and moved into the analysis method of the Analyzer class, in order to separate the concerns of reading and writing files and the logic of the agent.

4 Performance analysis and the influence of game settings

While playing with other students, we noticed that the parameters of the game almost seem to affect the outcome more than the implemented strategy. Therefore I decided to evaluate the performance of my agent in different type of game settings. The goal was to discover which type of settings affect the performance of my agent positively and which negatively[5]. I considered each parameter individually to discern the effect of each parameter without interference of others. The baseline for comparison were the standard settings. For each parameter I considered values above and below the standard value, but within the boundaries in which our agents will be tested (see Moodle answer by Timothy Harrison, 2025-11-20). Furthermore I considered two types of agents: Agent C uses an expansive, aggressive strategy similar to the strategy of my agent, Agent K plays much more defensively, i.e. it hardly expands, having few but well defended nodes and only attacks if it is strong enough. I played roughly 100-150 games against each agent for each setting, using the Tracker class to document the results (see the appendix for the detailed results).

I will consider the following hypotheses:

1. **Ring size**: Since my agent's strategy is strongly based on rapid expansion and maintaining as many nodes as possible, it is likely that my agent performs better on bigger rings. Bigger rings allow for more space to expand and a faster fernie growth through the node bonus.
2. **Node ownership bonus**: For the same reason as mentioned above, I predict a strong positive impact of a high node bonus on the performance of my agent.
3. **Fernie growth per turn**: I predict no effect of the growth rate per turn, since this rate affects both players, independently of the expansiveness of their strategy.
4. **Fernies at the beginning of the game**: I predict a positive effect of a higher number of starting fernies, since a higher number allows for faster take-off of the exponential growth through an expansive strategy.
5. **Visibility**: I predict no effect of the visibility, since visibility affects both players equally, and my agent does not use visibility in any particular way.

[5] Please note that the objective of this analysis was NOT to compare my agent with other students' agents, as all our agents were still in the development phase.

4.1 Ring size

The ring size appears to be a strong predictor for the performance of my agent. A lower than standard number of nodes (10) on the ring leads to a deteriorated performance against both agents: against Agent C the success rate dropped from 92% to 76% and against Agent K from 4% to 0%. In both cases a very high number of nodes (40) resulted in my agent winning 100% of games. This confirms hypothesis 1.

4.2 Fernie growth per turn

Against Agent C, as predicted, the growth rate per turn does not appear to strongly affect the performance. The success rates remains with ± 6 points of the result with standard settings. Contrary to my hypothesis, however, this parameter strongly affects the performance against Agent K. Reducing the growth rate to 5% halves the success rate (4% to 2%), while raising the growth rate to 20% improves my success rate to 20%. Raising the growth rate even further to 30% leads to a steep increase in my performance (99% success rate). In summary, against Agent K strongly raising the growth rate inverts the results from almost always losing to almost always winning.

A possible explanation for this effect against Agent K is that the total fernie growth rate is a multiplication of the growth rate and the node bonus (and other factors). By raising one factor (growth bonus) it also raises the effect of the other (node bonus). This effect is more pronounced against an agent that does not benefit as much from the node bonus such as Agent K.

4.3 Node ownership bonus

The effect of the node ownership bonus seem to behave similarly to the growth rate, only partially confirming my hypothesis. As expected a higher node bonus strongly improves the performance against the non-expanding Agent K, raising the success rate from 4% to over 95%. Against the expanding Agent C there is no effect at all (success rate of the standard value $\pm$ 2 points). This is probably explained by the fact that an equally expansive agent benefits from the node bonus in the same measure as my agent does.

4.4 Fernies at the beginning of the game

As predicted, a lower initial number of fernies (50) reduced the success rate of my agent against both Agent C (from 92% to 70%) and Agent K (from 4% to 3%). A higher number

(100) positively affected the results (92% to 93% for Agent C and 4% to 13% for Agent K). However, in both cases the difference only seems to be statistically significant in one direction. Curiously the effect is inverse for the two agents.

4.5 Visibility

I predicted no effect of the visibility on the performance, however, this hypothesis was entirely incorrect. Against Agent C the success rate of my agent plummets from 92% to 0% for a visibility range of 2 and to 36% for a visibility range of 8. Against Agent K the results are opposite, for a range of 2 my success rate rises from 4% to 95% and to 91% for a range of 8. There seems to be no obvious reason for this effect, further analysis is required to explain this behavior.

5 Conclusion

The Programmierpraktikum allowed me to dive into diverse subjects such as object oriented programming in Java, game strategies, basic AI theory, and last but not least moose and ants. I consider particularly valuable the experience I gained in iterative programming, through the process of planning, implementing, testing, analyzing the results, and repeating. I also learned about the importance of considering edge cases when testing my code. I only found many of the errors in my code by encountering very specific situations when playing against fellow students.

The final performance analysis of my agent uncovered interesting results about its behavior. Some of them were predictable, while others (such as the influence of the visibility range) were entirely surprising. This analysis would be a good starting point for further improving my agent by making it adapt to different game settings.

Literature

Adams, E. S. (2016). "Territoriality in ants (Hymenoptera: Formicidae): a review." In: *Myrmecological News* 23. Open Access, pp. 101–118. URL: https://myrmecologicalnews.org/cms/index.php?filename=volume23/mn23_101-118_printable.pdf&format=raw.

Eldridge, S. and W. Tschinkel (2001). "Mechanisms of population regulation in the fire ant Solenopsis invicta: an experimental study." In: *Journal of AnimalEcology* 70, pp. 355–369.

Ellis, S. and E. J. H. Robinson (2014). "Polydomy in red wood ants." In: *Insectes Sociaux* 61, pp. 111–122. DOI: 10.1007/s00040-013-0337-z.

Isle Royale Wolf-Moose Project (2024). *The Study*. URL: https://www.isleroyalewolf.org/the-study.

Murie, Adolph (1934). "The moose of Isle Royale." In: *University of Michigan - Miscellaneous Publications* 25.

Russell, Stuart and Peter Norvig (2010). *Artificial Intelligence - A Modern Approach*. 3rd ed. Pearson Education, Inc.

A Anhang

A.1 Results against Agent2

Description	Wins	Losses	Success rate	Source
AttackMax with local battles	80	43	65%	stats_10-11-2025_attackmax_local
AttackMax with triple and edge battles	80	22	78%	stats_10-11-2025_attackmax_with_edge_battles
AttackMax with additional 0.1% fernies for triple and edge battles	114	27	84%	stats_10-11-2025_edgebattle_1.1_localbattle_1.0

A.2 Results against itself - MixedStrategy

Description	Wins	Losses	Success rate	Source
MixedStrategy 0.2, 0.2, 0, 0.6 against 0.3, 0.3, 0, 0.4	90	19	83%	stats_10-11-2025_Mixed_2206vs3304
MixedStrategy 0.1, 0.1, 0, 0.8 against 0.2, 0.2, 0, 0.6	102	16	86%	stats_10-11-2025_Mixed_2206vs1108
MixedStrategy 0, 0, 0, 1 against 0.1, 0.1, 0, 0.8	125	8	94%	stats_10-11-2025_Mixed_1108_vs_00010

A.3 Results against Agent C

Description	Wins	Losses	Success rate	Source
Standard	121	10	92%	StudentC_Standard.txt
Ring size: 10	87	27	76%	StudentC_10n.txt
Ring size: 30	99	67	60%	StudentC_30n.txt
Ring size: 40	82	0	100%	StudentC_40n.txt
Fernie growth per turn: 5	105	17	86%	StudentC_5gpt.txt
Fernie growth per turn: 20	139	4	97%	StudentC_20gpt.txt
Fernie growth per turn: 30	111	11	90%	StudentC_30gpt.txt
Node ownership bonus: 1	105	12	90%	StudentC_1nb.txt
Node ownership bonus: 10	105	8	93%	StudentC_10nb.txt
Node ownership bonus: 20	111	10	92%	StudentC_20nb.txt
Starting fernies: 50	103	44	70%	StudentC_50sf.txt
Starting fernies: 100	96	7	93%	StudentC_100sf.txt
Visibility: 2	0	46	0%	StudentC_2v.txt
Visibility: 8	45	81	36%	StudentC_8v.txt

A.4 Results against Agent K

Description	Wins	Losses	Success rate	Source
Standard	4	105	4%	StudentK_Standard.txt
Ring size: 10	0	109	0%	StudentK_10n.txt
Ring size: 30	117	1	99%	StudentK_30n.txt
Ring size: 40	115	0	100%	StudentK_40n.txt
Fernie growth per turn: 5	2	128	2%	StudentK_5gpt.txt
Fernie growth per turn: 20	25	100	20%	StudentK_20gpt.txt
Fernie growth per turn: 30	115	1	99%	StudentK_30gpt.txt
Node ownership bonus: 1	2	112	2%	StudentK_1nb.txt
Node ownership bonus: 10	113	5	96%	StudentK_10nb.txt
Node ownership bonus: 20	108	0	100%	StudentK_20nb.txt
Starting fernies: 50	4	111	3%	StudentK_50sf.txt
Starting fernies: 100	17	112	13%	StudentK_100sf.txt
Visibility: 2	107	6	95%	StudentK_2v.txt
Visibility: 8	105	11	91%	StudentK_8v.txt